Lab 8 | Circular Sorted List

SortedType<int> - 27/27 checks passed

C++17 | Microsoft Visual C++ 19.44 | /W4 /WX | Exit status 0

Lab 8 | int test run

Actual executable output captured from: lab8.exe int | Exit status: 0

```
LAB 8 - TEMPLATE CIRCULAR SORTED LIST

=== SortedType<int> ===
[PASS] Constructor: empty list
[PASS] IsFull: memory available
[PASS] GetItem: empty list
[PASS] PrintReverse: empty list
[PASS] GetNextItem: empty-list exception
[PASS] DeleteItem / MakeEmpty: empty list
[PASS] PutItem: empty, tail, head; sorted traversal
Forward: 10 20 30
Reverse: 30 20 10
[PASS] PrintReverse: exact reversed order
[PASS] GetNextItem: tail wraps to head
[PASS] ResetList: restart at head
30 20 10
[PASS] PrintReverse preserves traversal position
[PASS] GetItem: find head, middle, tail
[PASS] GetItem: missing below, between, above
[PASS] PutItem: insert between existing nodes
[PASS] PutItem: duplicate; length is four
[PASS] DeleteItem: remove just one duplicate
[PASS] DeleteItem: missing values leave list unchanged
[PASS] DeleteItem: middle; traversal safely resets
[PASS] DeleteItem: head
[PASS] DeleteItem: tail
[PASS] PrintReverse: singleton
[PASS] DeleteItem: only node
[PASS] PutItem: all-equal circular list
[PASS] DeleteItem: all-equal list stays circular
[PASS] MakeEmpty: populated list and repeated call
[PASS] PutItem: reuse after MakeEmpty
[PASS] Destructor: populated list returns normally

RESULT: 27/27 checks passed; 0 failed.
```

Screenshot of captured executable output; complete transcript: test_output.txt.

All three runs: 82/82 checks passed. Required example: X Y Z -> Z Y X.

Lab 8 | Circular Sorted List

SortedType<double> - 27/27 checks passed

C++17 | Microsoft Visual C++ 19.44 | /W4 /WX | Exit status 0

Lab 8 | double test run

Actual executable output captured from: lab8.exe double | Exit status: 0

```
LAB 8 - TEMPLATE CIRCULAR SORTED LIST

=== SortedType<double> ===
[PASS] Constructor: empty list
[PASS] IsFull: memory available
[PASS] GetItem: empty list
[PASS] PrintReverse: empty list
[PASS] GetNextItem: empty-list exception
[PASS] DeleteItem / MakeEmpty: empty list
[PASS] PutItem: empty, tail, head; sorted traversal
Forward: 1.5 2.5 3.5
Reverse: 3.5 2.5 1.5
[PASS] PrintReverse: exact reversed order
[PASS] GetNextItem: tail wraps to head
[PASS] ResetList: restart at head
3.5 2.5 1.5
[PASS] PrintReverse preserves traversal position
[PASS] GetItem: find head, middle, tail
[PASS] GetItem: missing below, between, above
[PASS] PutItem: insert between existing nodes
[PASS] PutItem: duplicate; length is four
[PASS] DeleteItem: remove just one duplicate
[PASS] DeleteItem: missing values leave list unchanged
[PASS] DeleteItem: middle; traversal safely resets
[PASS] DeleteItem: head
[PASS] DeleteItem: tail
[PASS] PrintReverse: singleton
[PASS] DeleteItem: only node
[PASS] PutItem: all-equal circular list
[PASS] DeleteItem: all-equal list stays circular
[PASS] MakeEmpty: populated list and repeated call
[PASS] PutItem: reuse after MakeEmpty
[PASS] Destructor: populated list returns normally

RESULT: 27/27 checks passed; 0 failed.
```

Screenshot of captured executable output; complete transcript: test_output.txt.

All three runs: 82/82 checks passed. Required example: X Y Z -> Z Y X.

Lab 8 | Circular Sorted List

SortedType<char> - 28/28 checks passed

C++17 | Microsoft Visual C++ 19.44 | /W4 /WX | Exit status 0

Lab 8 | char test run

Actual executable output captured from: lab8.exe char | Exit status: 0

```
LAB 8 - TEMPLATE CIRCULAR SORTED LIST

=== SortedType<char> ===
[PASS] Constructor: empty list
[PASS] IsFull: memory available
[PASS] GetItem: empty list
[PASS] PrintReverse: empty list
[PASS] GetNextItem: empty-list exception
[PASS] DeleteItem / MakeEmpty: empty list
[PASS] PutItem: empty, tail, head; sorted traversal
Forward: A M Z
Reverse: Z M A
[PASS] PrintReverse: exact reversed order
[PASS] GetNextItem: tail wraps to head
[PASS] ResetList: restart at head
Z M A
[PASS] PrintReverse preserves traversal position
[PASS] GetItem: find head, middle, tail
[PASS] GetItem: missing below, between, above
[PASS] PutItem: insert between existing nodes
[PASS] PutItem: duplicate; length is four
[PASS] DeleteItem: remove just one duplicate
[PASS] DeleteItem: missing values leave list unchanged
[PASS] DeleteItem: middle; traversal safely resets
[PASS] DeleteItem: head
[PASS] DeleteItem: tail
[PASS] PrintReverse: singleton
[PASS] DeleteItem: only node
[PASS] PutItem: all-equal circular list
[PASS] DeleteItem: all-equal list stays circular
[PASS] MakeEmpty: populated list and repeated call
[PASS] PutItem: reuse after MakeEmpty
[PASS] Destructor: populated list returns normally
Assignment example X Y Z -> Z Y X
[PASS] Required X Y Z example

RESULT: 28/28 checks passed; 0 failed.
```

Screenshot of captured executable output; complete transcript: test_output.txt.

All three runs: 82/82 checks passed. Required example: X Y Z -> Z Y X.